

Machine Learning Final Project

George Brady

Student ID: 22335538

Part 1

Task 1.1:

To train and test my model I built datasets using python scripts. I built them using python scripts so that I could have complete control over everything in the dataset from what operations it uses, to how much of the dataset I want to have more complex or simple operations, to the size of the dataset.

As I wanted to build my model piece by piece so that I know exactly what is going on and also what is and is not working. I started off by making the datasets very simple and increasing the complexity after recording how the model works on the simple version. I also made all my datasets the same size of 5000 lines to make the evaluation fair and to give the model enough input to learn patterns but not overload it and make it take too long.

My first dataset was 5000 random single digit additions which looked like for example $7+6=13$. My next dataset was 5000 random multiple digit addition operations which looked like for example $11+51=62$. In my next dataset, I added multiple different types of operations like subtraction, multiplication, and division, where they are all equally likely. In my final dataset, I added brackets, so all of the operators were an equal 20% chance of happening and brackets with 2 random operators like for example $(3+2)*4=20$ had a 20% chance of happening. This helped me build my model overtime and find where exactly the model begins to break and where to make changes.

Task 1.2:

Since the only goal we have for our model is to get the singular correct answer when given an equation and there are not multiple correct answers with some being more correct than others, we only need a few metrics to evaluate the model. The main evaluation metric that I used to build my model and determine how well it is working is the accuracy of the model. When given a math equation, the model is either right, or wrong, and the accuracy measures the percentage of correct predictions the model made for equations in the dataset compared to incorrect predictions. To get the accuracy for every line in the test data I strip the test data to the input of the equation part for example $2+3$, I run the input through the model and compare the output of the first number that the model gives to the known output of the equation. Using the accuracy, I can get the error rate very simply as well, as the error rate is just 100% minus the accuracy.

To help me understand my accuracy metric, I also built a baseline predictor. My baseline predictor predicts the answer of the equation to be a random integer between the lowest and highest possible integer in the dataset. I made my baseline predictor to be this simplistic as I want to compare my model to a predictor that knows nothing about likelihoods, like multiplication is likely to lead to a higher answer, or two digit inputs are likely to lead to a two or three digit output. As my model recognizing these patterns and learning them, should be a valid improvement on the baseline.

Another metric that I used to help me build and evaluate the model is the log loss metric. This allows me to see how well the model is learning and how confident the model is. It also allows me to see how stable the model is and if the model is overfitted by comparing the log loss metric of the training vs the validation.

Task 1.3

To build the architecture of this model, I used my knowledge of the variables and the GPT architecture. I used testing to build the architecture step by step so that the model could solve math equations. For the tokenization of the model, I decided that single character tokenization is the best way for the purpose of math equations. Logically single character tokenization makes sense as the way to solve a math problem is to consider each character in the equation individually, they should not be grouped. This is especially true when the vocabulary is so small and fixed, being only the numbers 0-9 and the operators '+', '-', '*', '/', '(' and ')'.

For all the hyperparameters it is important to find the middle ground of having them large enough so that the model learns well enough, but not too large that the model takes too long to learn. The hyperparameter `n_embd`, is a very important parameter as I found during testing changing this parameter had the highest impact on the output for example for the single character addition dataset when my all my hyperparameters were small changing `n_embd` from 64 to 128 increased the accuracy of the model from 93.48% to 99.98%. Increasing this parameter has such a large impact because having a larger `n_embd` allows embedding vectors to carry more information about each token. I made my final choice for `n_embd` as `n_embd = 256` as if it was any higher than this my model becomes too slow and I still got great accuracy improvements of 65.00% to 77.20% when increasing my `n_embd` from 128 to 256 in my double-digit addition dataset.

For the number of heads and the number of layers, I chose 4 for both. I did this because the number of heads in the model basically tracks how much information is tracked in parallel, and the number of layers basically controls how many times the information comes through and is processed. It is important to have both of these large enough but they increase the training time of the model. To find the correct amount to set them at for the simplest dataset I set them both to one and saw a big jump in accuracy from 1 to 4 of 88.68% to 96.74%. This becomes more important as the dataset becomes more complex, and the patterns become more complex to track, so I set them both at 4 in the end.

For the block size of my model, I chose it to be 32. This is because the block size does not need to be too large for these small formulas. Since the length of each formula is not fixed, that means that we have to set the block size large enough to capture the full formula, and not have incomplete parts of a formula making a significant impact. When I tested different block sizes on the double digit addition dataset I found 32 to be the best as it gave an accuracy score of 77.20% whereas a block size of 16 was much worse giving an accuracy score of 67.00%, and 64 being computationally heavier and giving a slightly worse accuracy of 76.40%.

For solving math equations, I set the dropout to be 0.0. As in this case, there is no need for a dropout which introduces randomness into the program so that the model does not become overfitted. This is because in math there is always one correct answer every time. This is an important variable that makes no change to the computation time and for my complicated multiple operations with no brackets dataset, having the dropout at 0 gives an accuracy of 48.70% compared to a dropout of 0.2 which gives an accuracy of 17.03%.

For the cost function, I kept the log loss function as I decided to use single character tokenization. This cost function measures how likely from the model's point of view the next character is likely to be correct. There are other ways to do the cost function, but I thought that this is the best way to do it for my model where my model runs completely off of left to right single characters.

Task 1.4

The way that I went about exploring a range of arithmetic operations, finding out which one's work and how to set the parameters. I started off with the most simplistic dataset and extremely low parameters. I upped the parameters until the model worked well enough for that dataset and then made the dataset harder.

To start off, I used a dataset that just involved addition with single digit numbers, so in the dataset only the numbers 0-9 were added to each other to get the answer. This dataset had 5000 of these equations as I have explained before gives me a more fair comparison between datasets when I change them and keep the length. This dataset once I had set the parameters was a very easy dataset for the model to learn and gave an accuracy of 99.98% and an error rate of 0.02%. This is because in single digit addition each character can be treated completely individually and the model just has to learn the pattern of how much each character is worth and where to put them. Some examples of the output of the program are $7+8=15$, $3+0=3$, $2+5=7$, $2+8=10$, $9+5=14$. I could not find the 0.02% examples of where the model did not work well as it is such a low rate for this dataset.

The next dataset that I used is a double-digit character addition set. I decided to use this dataset as before I added other operations like addition and multiplication, I wanted to see how the program performs on a dataset that has double digits in it like 76, 41 etc. This is quite a bit harder for the model as the model has to learn the patterns between individual numbers in a larger number for

example the 4 in 41 represents four tens and not a 4, while still learning addition on top of that. When I introduced this dataset, I got an accuracy score of 77.20% with an error rate of 23.6%. You can see that this is quite a big decline from the single character addition, but the model is still correct most of the time and is clearly learning valid patterns in a reasonable amount of time. It is much better than the baseline predictor which randomly predicts answers between the smallest and largest number given in this addition of $0+0=0$ and $99+99=198$ which gave a 0.8% accuracy score. The difference between 77.2% and 0.8% shows that even though the model is not performing perfectly, it is still learning a lot. Examples of the model given correct answers are $20+83=103$, $73+96=169$, $68+34=102$, $47+14=61$ and an example of the model giving the incorrect answer are $30+94=125$.

For the next dataset I added multiple operators, each operator addition, subtraction, division and multiplication get an equal chance of being the operator in the equation. This now tests how the model is at learning multiple operators at once and not just one individually. The model has an accuracy of 48.70% and an error rate of 51.3% which is getting lower again but is much bigger than the baseline predictor which predicts random integers from the smallest to the largest integer and gives an accuracy of 0.0%. The model is learning the different operators at different rates. These rates are: for '+' the accuracy is 38.52%, for '-' the accuracy is 45.54%, for '*' the accuracy is 7.96%, for '/' the accuracy is 94.24%. The reason that these numbers are vastly different is because the way that the dataset that the model is trained on is set up. For example, division has a really high accuracy but that is because the actual division dataset has much more repeating patterns and equations than the multiplication dataset because I ensure that the division does not leave a remainder as I am only considering integer equations for my model. The same with subtraction which has a better accuracy than addition, this is because I am not considering negative numbers in my dataset and the possible subtraction equations are less than addition. Some examples of equations that my model got correct are $38-5=33$, $12/2=6$, $4+39=43$ and some examples of equations that my model got wrong are $67*52=3284$, $43+33=86$ and $23*20=440$.

I added brackets to my last dataset so addition, subtraction, multiplication, division and brackets were all equally likely to happen. Brackets involve 2 sets of operators and look like $(3+2)*4=20$ for example. When I added brackets, the accuracy went down to 35.69% with an error rate of 64.31%. This is down a bit again but comparing it to my baseline predictor of any integer between the minimum and maximum output, which gives 0.0%, still shows that the model is learning valid patterns in the data. The individual accuracy for each operation is: for '+' the accuracy is 22.70%, for '-' the accuracy is 32.31%, for '*' the accuracy is 2.13%, for '/' the accuracy is 67.72% and for equations involving brackets the accuracy is 8.43%. You can see that the accuracy for all of the original operators goes down as the model tries to learn a new operator and the addition of brackets at this stage of complexity has quite a low accuracy. Some correct example outputs of the model are $(35-20)/5=3$, $51/3=17$ and $20+28=48$ and some examples of incorrect outputs from the model are $(46-17)/3=11$ and $70*69=4820$.

Part 2

Task 2.1

Once again to train and test my model, I built the datasets using python scripts so that I could have complete control over everything in the dataset from what operations it uses, to how much of the dataset I want to have more complex or simple operations, to the size of the dataset. And I once again make my datasets the same size of 5000 lines to make the evaluation fair and to give the model enough input to learn patterns but not overload it and make it take too long.

This time I only built two datasets as there are only two main patterns that I want to see the machine learn, boolean logic and multi boolean logic. The first dataset has an equal likelihood that every operator will be 'NOT', 'AND', 'OR' or 'XOR'. For NOT it is an equal likelihood that the other input value will be 'True' or 'False' and for the others it will be of equal likelihood that both values are True or False. In simple terms, I am testing if the model can do each operator in its simplest form on booleans. Examples of the inputs look like "False OR True = True", "NOT False = True", "True AND False = False" and "True XOR True = False".

For my next and final dataset, I make it able to handle two operators in one equation and add brackets around one of these operators. This dataset is made to teach the model how to complete equations that have multiple operators and variables in them at once with brackets dictating in what order they should be done. Examples of the dataset for this are "False AND True = False", "(True AND True) AND False = False", "False XOR (False OR True) = True".

Task 2.2

The evaluation metrics that I used for Boolean GPT are pretty much the same as for math GPT for the same reasons. Since the only goal we have for our model is to get the singular correct answer "True" or "False" when given an equation and there are not multiple correct answers with some being more correct than others, we only need a few metrics to evaluate the model. The main evaluation metric that I used to build my model and determine how well it is working is the accuracy of the model. When given a Boolean equation, the model is either right or wrong and the accuracy measures the percentage of correct predictions the model made for equations in the dataset compared to incorrect predictions. To get the accuracy for every line in the test data, I strip the test data to the input of the equation part for example "True AND False", I run the input through the model and compare the first word in the output the model gives to the known output of the equation. Using the accuracy, I can get the error rate very simply as well, as the error rate is just 100% minus the accuracy.

To help me understand my accuracy metric, I also built a baseline predictor. My baseline predictor predicts the answer of the equation to be a "True" or "False" randomly. I made my baseline predictor to be this simplistic as I want to compare my model to a predictor that knows nothing about the likelihoods of the output being "True" or "False" allowing me to evaluate if the model is recognizing correct patterns in the dataset.

Another metric that I used to help me build and evaluate the model is the log loss metric. This allows me to see how well the model is learning and how confident the model is. It also allows me to see how stable the model is and if the model is overfitted by comparing the log loss metric of the training vs the validation.

Task 2.3

To build the architecture of this model, I used my knowledge of the variables used and the GPT architecture. I used testing to build the architecture so that the model could solve boolean equations. For the tokenization of the model, I decided that single character tokenization is the best way for the purpose of boolean equations. This is because the dataset has a very small number of unique variables 'NOT', 'AND', 'OR', 'XOR', 'True', 'False', '=', '(', and ')', and since it is such a small set and they always appear in that exact form the model should be very capable of learning them. I also saw no need to change the tokenization to a word based one as when I tested my model on the simple dataset of just the single boolean equations it achieved an accuracy of 100%, meaning it got every answer in the test set correct and learnt these words fully over the 5,000 line dataset. I do however use a simplification in my accuracy function to account for single character tokenization outputs where I only take the first word produced by the model and ignore anything that the model produces after that.

For all the hyperparameters it is important to find the middle ground of having them large enough so that the model learns well enough but not too large that the model takes too long to learn. The first hyperparameter that I found to make a large difference is the block_size. When I tested different block_size values on my complicated dataset, I got an overall accuracy of 96.43% for a block size of 32, an accuracy of 98.02% on a block size of 64, and an accuracy of 98.81% on a block size of 128. I chose my final block size to be 128 as it has the best accuracy score and a much better accuracy score for the more complicated operators like for example XOR which went from 93.47% at a block_size of 32 to 97.49% at a block size of 128. The block size is so important for this model as the input lines are a very large size especially when doing single character tokenization and looking at more lines at once decreases the negative impacts of one of the lines being cut off and incomplete.

The hyperparameter n_embd is also an important parameter as I found during testing. Where when I changed my n_embd vector from 64 to 128 to 256, the overall accuracy of the model went from 91.47% to 98.81% to 99.01%. This parameter has such an impact because having a larger n_embd allows embedding vectors to carry more information about each token. So, the final n_embd vector that I choose is 256 as this gives the highest accuracy and is not too computationally heavy. For the number of heads and the number of layers, I chose 2 for both. This is because when I tested different values for n_head and n_layer from 1 to 2 to 4. I got an accuracy of 87.10% when the number of heads and layers is 1, I got an improved accuracy of 99.21% when the number of heads and layers is 2 and I got a slightly worse overall accuracy when the number of heads and layers is 4 of 99.01%. The accuracy of the model peaks at n_layers = 2 and n_heads = 2 for this model as

boolean logic is very simple when it comes to the patterns that the model has to learn, and the model does not need extra layers of deep reasoning to learn these patterns.

Once again I set the dropout to be 0.0. As in this case, there is no need for a dropout which introduces randomness into the program so that the model does not become overfitted. This is because in boolean equations there is always one correct answer every time. This is an important variable that makes no change to the computation time and for my dataset having the dropout at 0 gives an accuracy of 99.21% compared to a dropout of 0.2 which gives an accuracy of 94.05%.

Once again, for the cost function I kept the log loss function as I decided to use single character tokenization. This cost function measures how likely from the model's point of view the next character is likely to be correct. There are other ways to do this, but I thought that this is the best way to do it for my model where my model runs completely off of left to right single characters.

Task 2.4

The way that I went about exploring a range of arithmetic operations and finding out which ones work and how to set the parameters. Is I started off with the simple dataset where there is only one operator and likelihood that every operator will be 'NOT', 'AND', 'OR' or 'XOR' is the same. So, an example of what lines look like in this model are "NOT True = False", "True AND True = True". This dataset is very simplistic for a GPT model as there are very few possible combinations of inputs and only two possible outputs. My model when given this dataset has a 100% accuracy rate and a 0% error rate for all of the operators. This means the model learnt them all well and didn't get any of the answers incorrect in the testing data. When comparing this to the baseline predictor which predicts true or false randomly and has an accuracy rate of 47.42%, we can see how good a 100% accuracy score is. Sample outputs from this dataset and the model are "False XOR False = False", "True OR True = True" and "NOT True = False".

For my more complicated and final dataset, I have the same logic as the first dataset 50% of the time. The other 50% of the time there are two different operations with brackets around one of them. This dataset teaches the model to deal with multiple Boolean operations in one equation and to learn that the operation inside of the brackets should be done first. Example, lines in this dataset are "True XOR True = False", "(True AND False) AND True = False" and "False OR (True AND True) = True".

When I gave the model this dataset, it had an overall accuracy of 99.21%, which is an error rate of 0.79%. For all equations with "AND" and "NOT" in the testing data the model achieved a 100% accuracy rate. For all equations with the "OR" operator in it, the model achieved an accuracy rate of 99.15%. For all equations with the "XOR" operator in it, the model achieved an accuracy rate of 98.49%. And for all equations that had two operators and brackets in them, the model achieved an accuracy rate of 98.78%. This shows that the model learnt all of the operations very well, especially when compared to the baseline predictor which randomly predicts true or false which gave an

accuracy of 49.60% compared to the accuracy of our model which is 99.21%. Some example outputs that my model gave are “(False OR True) AND True = True”, “(True OR False) XOR True = False” and “True OR (True XOR False) = True”.

Part 3

Overall, Math GPT is very similar in a lot of ways to Boolean GPT in terms of the way in which I approached making both of them and their core structures, but Boolean GPT is a lot simpler and there are still a few differences between them. Firstly, the things that I kept the same are the tokenization of the actual model, the evaluation metrics, the `n_embd` and the dropout. All these structures and values after testing I found were the most optimal for both models. The main things that I changed between the models is how I could compute accuracy in math GPT it had to be on numbers and in Boolean GPT it had to be based on stripped boolean values. I change the `n_head` and `n_layer` from 4 in math GPT to 2 in Boolean GPT as Boolean GPT is based off a much simpler dataset in terms of possible combinations and outputs. I had to change the `block_size` dramatically from math GPT where it was 32 to Boolean GPT where it is 128, which is four times the length because the number of tokens per equation in Boolean GPT is much more than in math GPT.

In terms of evaluating both models, Boolean GPT performed much better on the final dataset with Math GPT having a much lower accuracy of 35.69% and higher error rate of 64.31%, compared to Boolean GPT which has an accuracy of 99.21% and error rate of 0.79%. This is mainly because math GPT has a dataset that spans a much wider range of inputs variables and outputs even though they have a similar number of operators and bracket logic, with something like multiplication having possible inputs of 0-99 and possible outputs of 0-9801. While Boolean logic only has possible input variables and outputs of true and false. So, Math GPT fails on operations with large inputs and outputs. This is confirmed not just by comparing it to Boolean GPT but also that multiplication has the lowest accuracy out of the operations. And because Boolean GPT does not have a large range of inputs or outputs, it has very little failures. To solve the issue of math GPT not performing as well due to complexity of the dataset in the future, I would need to increase the hyperparameters like `n_embd`, `n_head`, `n_layer` and number of iterations. This would require much more time and computing power.

Appendix:

Code for the maths GPT:

```
import torch
import torch.nn as nn
from torch.nn import functional as F
import random

batch_size = 16
block_size = 32
max_iters = 5000
```

```

eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 256
n_head = 4
n_layer = 4
dropout = 0.0
# -----

torch.manual_seed(1337)

# wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
with open('mathsDataset.txt', 'r', encoding='utf-8') as f:
    text = f.read()

# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)
# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string

# Train and test splits
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]

# data loading
def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():

```

```

out = {}
model.eval()
for split in ['train', 'val']:
    losses = torch.zeros(eval_iters)
    for k in range(eval_iters):
        X, Y = get_batch(split)
        logits, loss = model(X, Y)
        losses[k] = loss.item()
    out[split] = losses.mean()
model.train()
return out

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # input of size (batch, time-step, channels)
        # output of size (batch, time-step, head size)
        B, T, C = x.shape
        k = self.key(x) # (B,T,hs)
        q = self.query(x) # (B,T,hs)
        # compute attention scores ("affinities")
        wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
        wei = F.softmax(wei, dim=-1) # (B, T, T)
        wei = self.dropout(wei)
        # perform the weighted aggregation of the values
        v = self.value(x) # (B,T,hs)
        out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)
        return out

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

```

```

def __init__(self, num_heads, head_size):
    super().__init__()
    self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
    self.proj = nn.Linear(head_size * num_heads, n_embd)
    self.dropout = nn.Dropout(dropout)

def forward(self, x):
    out = torch.cat([h(x) for h in self.heads], dim=-1)
    out = self.dropout(self.proj(out))
    return out

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

```

```

class GPTLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # better init, not covered in the original GPT video, but important, will cover in followup video
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
        x = tok_emb + pos_emb # (B,T,C)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x) # (B,T,C)
        logits = self.lm_head(x) # (B,T,vocab_size)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)

```

```

return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        # get the predictions
        logits, loss = self(idx_cond)
        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)
        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)
        # pick the highest probability everytime
        idx_next = torch.argmax(probs, dim=-1, keepdim=True)
        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
    return idx

model = GPTLanguageModel()
m = model.to(device)
# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for iter in range(max_iters):

    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

```

```

# generate from the model
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))
#open('more.txt', 'w').write(decode(m.generate(context, max_new_tokens=10000)[0].tolist()))

@torch.no_grad()
def evaluation_loss_for_if_answer_correct(model, text):
    model.eval()
    # set the parameters for testing how much are correct
    model_is_correct = 0
    total_tested = 0
    dummy_is_correct = 0
    print_20_sentences = 0
    # loop through each line and calculate for each equation
    for current_equation in text.splitlines():
        # dont calculate if there is no equal sign and add 1 to the
        # total as it has to be wrong
        if '=' not in current_equation:
            total_tested = total_tested + 1
            continue
        # split into the equation and the answer using the = sign
        equation, equation_answer = current_equation.split('=')
        final_equation = equation + '='
        final_equation_answer = equation_answer.strip()
        # encode the equation part
        encoded_equation = torch.tensor([encode(final_equation)], dtype=torch.long, device=device)
        # get the same amount of answers as the amount of answers to the equation we have
        encoded_answers = model.generate(encoded_equation, max_new_tokens=len(final_equation_answer))
        # decode the answers to get the actual answers
        decoded_answers = decode(encoded_answers[0].tolist())
        # get just the answer part and get rid of the equation part
        answers_from_GPT = decoded_answers[len(final_equation):]
        # print out the first 10 useful equations
        if 0 < print_20_sentences <= 20:
            print(f"{final_equation}{answers_from_GPT}, the models answer is {final_equation_answer}")
            print_20_sentences = print_20_sentences + 1
        # add to the correct variable if it is correct
        if (final_equation_answer == answers_from_GPT):
            model_is_correct = model_is_correct + 1
        # add to the total no matter what
        total_tested = total_tested + 1
        # get the dummy answer as a random number between the max and min anwwer

```

```

dummys_answer = random.randint(0, 970299) # 198 for addition only, and 9801 for multiplication as
well, 970,299 with brackets
# if the dummy answer is right add to the dummy variable
if (str(dummys_answer) == final_equation_answer):
    dummy_is_correct = dummy_is_correct + 1
model.train()
accuracy_of_the_model = model_is_correct/total_tested
accuracy_of_the_dummy = dummy_is_correct/total_tested
return accuracy_of_the_model, accuracy_of_the_dummy

# get the test data
testing_data = decode(val_data.tolist())
# get the accuracy of the test data
total_accuracy, total_dummy_accuracy = evaluation_loss_for_if_answer_correct(model, testing_data)
print(f"the accuracy of the model is: {total_accuracy:.2%}")
print(f"the accuracy of the dummy model is: {total_dummy_accuracy:.2%}")

# loop through all of the operators that we want to test individually
for current_operator in ['+', '-', '*', '/', '']:
    # create add the equations with the current operators to the list
    current_operator_equations = ""
    for current_equation in testing_data.splitlines():
        if current_operator in current_equation:
            current_operator_equations = current_operator_equations + current_equation + '\n'
        if current_operator_equations == "":
            print(f"no {current_operator} in this dataset")
        else:
            current_operator_accuracy, current_operator_dummy_accuracy =
            evaluation_loss_for_if_answer_correct(model, current_operator_equations)
            print(f"for {current_operator} the accuracy is {current_operator_accuracy:.2%}")

```

Outputs for the maths GPT:

- 60+9=69, the models answer is 69
- 21+10=31, the models answer is 31
- 61+93=154, the models answer is 134
- 21-3=18, the models answer is 18
- 25-14=11, the models answer is 11
- 76-9=67, the models answer is 57
- 39*78=3042, the models answer is 3032

- $94 \times 23 = 2162$, the models answer is 2132
- $77 / 1 = 77$, the models answer is 77
- $26 / 13 = 2$, the models answer is 2
- $(42 / 42) + 92 = 93$, the models answer is 11
- $(93 / 93) \times 72 = 72$, the models answer is 40

Code for the maths GPT:

```
import torch
import torch.nn as nn
from torch.nn import functional as F
import random

batch_size = 16
block_size = 128
max_iters = 5000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 256
n_head = 2
n_layer = 2
dropout = 0.0
# -----

torch.manual_seed(1337)

# wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
with open('booleanDataset.txt', 'r', encoding='utf-8') as f:
    text = f.read()

# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)
# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string

# Train and test splits
```

```

data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]

# data loading
def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # input of size (batch, time-step, channels)

```

```

# output of size (batch, time-step, head size)
B,T,C = x.shape
k = self.key(x) # (B,T,hs)
q = self.query(x) # (B,T,hs)
# compute attention scores ("affinities")
wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
wei = F.softmax(wei, dim=-1) # (B, T, T)
wei = self.dropout(wei)
# perform the weighted aggregation of the values
v = self.value(x) # (B,T,hs)
out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)
return out

```

```

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

```

```

    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)

```

```

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out

```

```

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

```

```

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

```

```

    def forward(self, x):
        return self.net(x)

```

```

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

class GPTLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # better init, not covered in the original GPT video, but important, will cover in followup video
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape

```

```

# idx and targets are both (B,T) tensor of integers
tok_emb = self.token_embedding_table(idx) # (B,T,C)
pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
x = tok_emb + pos_emb # (B,T,C)
x = self.blocks(x) # (B,T,C)
x = self.ln_f(x) # (B,T,C)
logits = self.lm_head(x) # (B,T,vocab_size)

if targets is None:
    loss = None
else:
    B, T, C = logits.shape
    logits = logits.view(B*T, C)
    targets = targets.view(B*T)
    loss = F.cross_entropy(logits, targets)

return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        # get the predictions
        logits, loss = self(idx_cond)
        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)
        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)
        # pick the highest probability everytime
        idx_next = torch.argmax(probs, dim=-1, keepdim=True)
        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
    return idx

model = GPTLanguageModel()
m = model.to(device)

# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

```

```

for iter in range(max_iters):

    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

    # generate from the model
    context = torch.zeros((1, 1), dtype=torch.long, device=device)
    print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))
    # open('more.txt', 'w').write(decode(m.generate(context, max_new_tokens=10000)[0].tolist()))

@torch.no_grad()
def evaluation_loss_for_if_answer_correct(model, text):
    model.eval()
    # set the parameters for testing how much are correct
    model_is_correct = 0
    total_tested = 0
    dummy_is_correct = 0
    print_10_sentences = 0
    # loop through each line and calculate for each equation
    for current_equation in text.splitlines():
        # dont calculate if there is no equal sign and add 1 to the
        # total as it has to be wrong
        if '=' not in current_equation:
            total_tested = total_tested + 1
            continue
        # split into the equation and the answer using the = sign
        equation, equation_answer = current_equation.split('=')
        final_equation = equation + '='
        final_equation_answer = equation_answer.strip()
        # encode the equation part
        encoded_equation = torch.tensor([encode(final_equation)], dtype=torch.long, device=device)
        # allow only the max length of 6 as false produces an answer of length 6

```

```

encoded_answers = model.generate(encoded_equation, max_new_tokens=6)
# decode the answers to get the actual answers
decoded_answers = decode(encoded_answers[0].tolist())
# get just the answer part and get rid of the equation part
parts = decoded_answers.split('=')
# only is the answer part is split into parts by the equals
if len(parts) >= 2:
# get the first word after generating and dont let the model continue
answer_part = parts[1].strip()
answer_without_continuing = answer_part.split()
if answer_without_continuing:
answers_from_GPT = answer_without_continuing[0]
else:
"""
else:
answers_from_GPT = ""
# print out the first 10 useful equations
if 0 < print_10_sentences <= 10:
print(f'{final_equation} {final_equation_answer}, the models answer is {answers_from_GPT}')
print_10_sentences = print_10_sentences + 1
# add to the correct variable if it is correct
if (final_equation_answer == answers_from_GPT):
model_is_correct = model_is_correct + 1
# add to the total no matter what
total_tested = total_tested + 1
# pick randomly between true and false
dummys_answer = random.choice(["True", "False"])
# if the dummy answer is right add to the dummy variable
if (str(dummys_answer) == final_equation_answer):
dummy_is_correct = dummy_is_correct + 1
model.train()
accuracy_of_the_model = model_is_correct/total_tested
accuracy_of_the_dummy = dummy_is_correct/total_tested
return accuracy_of_the_model, accuracy_of_the_dummy

# get the test data
testing_data = decode(val_data.tolist())
# get the accuracy of the test data
total_accuracy, total_dummy_accuracy = evaluation_loss_for_if_answer_correct(model, testing_data)
print(f'the accuracy of the model is: {total_accuracy:.2%}')
print(f'the accuracy of the dummy model is: {total_dummy_accuracy:.2%}')

```

```

# loop through all of the operators that we want to test individually
for current_operator in ['AND', 'OR', 'NOT', 'XOR', '']:
# create add the equations with the current operators to the list
current_operator_equations = ""
for current_equation in testing_data.splitlines():
if current_operator in current_equation:
current_operator_equations = current_operator_equations + current_equation + '\n'
if current_operator_equations == "":
print(f"no {current_operator} in this dataset")
else:
current_operator_accuracy, current_operator_dummy_accuracy =
evaluation_loss_for_if_answer_correct(model, current_operator_equations)
print(f"for {current_operator} the accuracy is {current_operator_accuracy:.2%}")

```

Outputs for the maths GPT:

- NOT True = False, the models answer is False
- NOT False = True, the models answer is True
- (True OR False) XOR True = False, the models answer is False
- True XOR False = True, the models answer is True
- (False OR True) AND True = True, the models answer is True
- True AND False = False, the models answer is False
- False AND (True OR False) = False, the models answer is False